

BudgetFlowFusion

Dokumentacja analityczno-projektowa i użytkowa

Zakres dokumentu

- analiza dziedziny i projektu
- architektura aplikacji
- diagramy i opis modelu danych
- procesy biznesowe
- API backendu
- instrukcja użytkownika
- uruchomienie, testy i utrzymanie

Spis treści

Dokumentacja techniczna BudgetFlowFusion

Wprowadzenie

Dokumentacja analityczno-projektowa

Architektura systemu

Model danych

Procesy biznesowe

API backendu

Dokumentacja użytkownika

Uruchomienie i testy

Utrzymanie i słownik pojęć

Dokumentacja techniczna BudgetFlowFusion

Lista rozdziałów

Wprowadzenie

Cel systemu

BudgetFlowFusion jest aplikacją do obsługi zakupów, budżetów, dofinansowań, planów zamówień publicznych, wniosków i rozliczeń w kole naukowym. System zastępuje rozproszone arkusze i ustalenia ręczne jednym modelem danych oraz jednym procesem operacyjnym.

Zakres funkcjonalny

System obsługuje:

- rejestrację i logowanie użytkowników,
- rozróżnienie zwykłego członka i skarbnika,
- katalog przedmiotów oraz zgłaszanie sklepów,
- listy zakupów prowadzone dla konkretnego sklepu i finansowania,
- budżety koła, sekcji i dofinansowań,
- roczne plany zamówień publicznych powiązane z dofinansowaniem,
- wnioski o zamówienie z alokacją środków i pozycji planu,
- finalizację wniosku wraz ze snapshotem danych,
- rozliczenia, faktury i linie rozliczeniowe.

Role

Student Użytkownik aplikacji. Może dodawać przedmioty, zgłaszać sklepy i pracować na otwartych listach zakupów.

ProjectFinanceManager Skarbnik. Jest powiązany ze studentem przez Student.project_finance_manager_id. Zarządza finansami, planami, listami, wnioskami i rozliczeniami.

Granice systemu

System nie integruje się z SAP ani z zewnętrznym systemem księgowym. Pole Student.is_in_sap ma charakter informacyjny. Dokumenty i faktury są odwzorowane jako dane w bazie, bez obsługi uploadu pełnych plików dokumentów poza nazwą pliku rozeznania rynku.

Główna reguła finansowa

Źródłem pieniędzy w systemie jest Funding. Budżety sekcji i koła są wyliczane z dofinansowań oraz zarezerwowanych/wykorzystanych kwot.

```
Funding.funding_price
- Funding.spent_money
- SUM(PurchaseRequestFundingAllocation.allocated_amount)
= dostępne środki po rezerwacjach
```

Główna reguła projektowa

Plan zamówień publicznych jest przypisany do jednego dofinansowania. Wniosek rezerwuje środki i zużywa wskazane pozycje planu. Finalizacja zapisuje snapshot, żeby późniejsze zmiany planu lub dofinansowania nie zmieniły historii dokumentu.

Dokumentacja analityczno-projektowa

Problem

Koło naukowe prowadzi zakupy z wielu źródeł finansowania. Każdy zakup musi być przypisany do sekcji/projektu, dofinansowania, pozycji planu zamówień publicznych oraz późniejszego rozliczenia. Bez systemu dane są rozproszone między listami zakupów, arkuszami budżetowymi, wnioskami i fakturami.

Cel projektowy

Celem systemu jest utrzymanie spójnego przebiegu:

```
dofinansowanie
-> plan zamówień publicznych
-> lista zakupów
-> wniosek
-> finalizacja
-> rozliczenie
-> faktura
```

Aplikacja ma pilnować:

- dostępności środków,
- zgodności wniosku z pozycją CPV,
- wykorzystania planu publicznego,
- powiązania koszyków z wnioskiem,
- historii finalizacji,
- statusów faktur i rozliczeń.

Aktorzy

Aktor	Odpowiedzialność
Członek koła	przedmioty, sklepy, wkład do list zakupów
Skarbnik	budżety, finansowania, plany, wnioski
Obsługa rozliczeń	faktury, statusy, kwoty rzeczywiste

Obsługa rozliczeń nie jest osobnym typem konta w modelu. Jej czynności są realizowane przez moduł Settlement i Invoice.

Model ról

```
Student
|
| project_finance_manager_id IS NULL
v
zwykły członek

Student
```

```
|  
| project_finance_manager_id -> ProjectFinanceManager  
v  
skarbnik
```

Frontend ukrywa funkcje zależnie od roli. Backend dodatkowo sprawdza część operacji skarbnika, np. zarządzanie kategoriami, sklepami i finansami.

Wymagania funkcjonalne

- RF-01. Użytkownik może się zarejestrować i zalogować.
- RF-02. System rozróżnia zwykłego członka i skarbnika.
- RF-03. Członek może dodać przedmiot do katalogu.
- RF-04. Członek może zgłosić sklep. Skarbnik może sklep zaakceptować, edytować albo odrzucić.
- RF-05. Skarbnik może tworzyć dofinansowania wraz z zadaniami budżetowymi.
- RF-06. Skarbnik może utworzyć plan zamówień publicznych dla dofinansowania.
- RF-07. Skarbnik może tworzyć listy zakupów i przypisywać je do sklepu, dofinansowania oraz grupy zakupowej.
- RF-08. Użytkownicy mogą dodawać wkład do otwartych list zakupów.
- RF-09. Skarbnik może zamknąć listę i użyć jej przy wniosku.
- RF-10. Skarbnik może utworzyć wniosek o zamówienie z alokacją środków.
- RF-11. Wniosek może korzystać z jednej lub wielu pozycji planu.
- RF-12. Przekroczenie pozycji planu wymaga uzasadnienia.
- RF-13. Finalizacja wniosku zapisuje dane historyczne w snapshotach.
- RF-14. Rozliczenie wymaga obsługi faktur i kwot rzeczywistych.

Wymagania niefunkcjonalne

- Backend udostępnia API REST w formacie JSON.
- Frontend działa jako SPA w Vue 3.
- Dane trwale są przechowywane w PostgreSQL.
- Modele bazodanowe są definiowane w SQLAlchemy.
- Testy backendu używają SQLite in-memory.
- Dokumentacja jest generowana z plików doc/*.rst przez doc_gen.py.

Decyzje projektowe

Budżet wynika z dofinansowań

ProjectBudget.total_budget i AssociationBudget.total_budget nie powinny być traktowane jako niezależne źródło prawdy. W raportowaniu kwoty są liczone z tabeli Funding i rezerwacji wniosków.

Plan jest przy dofinansowaniu

PublicPurchasePlanList ma relację 1:1 z Funding. Dzięki temu pozycje CPV są kontrolowane w kontekście konkretnego źródła pieniędzy.

Wniosek rezerwuje środki

Tabela `PurchaseRequestFundingAllocation` przechowuje kwoty zarezerwowane na wniosek. Jeżeli wniosek nie ma rekordów alokacji, backend używa pola legacy `PurchaseRequest.budget_allocated_for_the_order` jako fallback.

Finalizacja zapisuje snapshot

`PurchaseRequestFinalizationSnapshot` przechowuje skopiowane dane planu i finansowania: CPV, numer planu, nazwę finansowania, kwoty netto, brutto i EUR. Snapshot jest potrzebny, ponieważ plan albo finansowanie mogą zostać później zmienione.

Koszty przechowuje wkład użytkowników

`ShopPurchaseListItem` przechowuje łączną ilość przedmiotu na liście.

`ShopPurchaseListItemContribution` przechowuje ile dodał konkretny student. To pozwala zwykłemu członkowi usuwać tylko własny wkład.

Zakres ryzyka

Największe ryzyka projektowe:

- podwójne liczenie kwot z dofinansowań i wniosków,
- niespójność planu CPV po edycji lub usunięciu pozycji,
- brak pełnej autoryzacji backendowej dla każdej operacji finansowej,
- różnice między PostgreSQL runtime i SQLite w testach,
- logika biznesowa umieszczona bezpośrednio w endpointach.

Architektura systemu

Widok warstwowy



Backend

Katalog: backend.

Najważniejsze pliki:

- backend/run.py - uruchomienie Uvicorn na porcie 8080,
- backend/src/__init__.py - konfiguracja FastAPI, CORS, silnik bazy, lifecycle i migracja,
- backend/src/relations.py - encje SQLAlchemy,
- backend/src/routes - moduły endpointów.

Rejestracja tras

backend/src/__init__.py tworzy globalny obiekt app. Import src.routes łączy wszystkie moduły z katalogu tras i rejestruje dekoratory @app.get, @app.post, @app.patch, @app.delete.

```
src.__init__
|
+-- app = FastAPI(...)
+-- engine = create_engine(DATABASE_URL)
+-- get_session()
+-- prepare_database()
+-- import src.routes
|
+-- login_register_routes
+-- members_routes
+-- items_routes
+-- shops_routes
+-- lists_routes
+-- fundings_routes
+-- public_purchase_plans_routes
```



```
+-- purchase_request_routes
+-- settlements_routes
```

Moduły backendu

login_register_routes.py Rejestracja i logowanie.

members_routes.py Lista studentów.

items_routes.py Katalog przedmiotów, statusy pending/approved/rejected, grupowanie.

shops_routes.py Sklepy, zgłaszanie, akceptacja, edycja i odrzucenie.

categories_subcategories_routes.py Kategorie, podkategorie i przypisanie CPV.

lists_routes.py Listy zakupów, pozycje, wkład użytkowników, zamykanie i otwieranie.

fundings_routes.py Dofinansowania, zadania budżetowe i dostępne środki.

public_purchase_plans_routes.py Budżety, dashboard, listy planów i pozycje planu publicznego.

purchase_request_routes.py Wnioski, alokacje finansowania, pozycje planu, finalizacja i linie rozliczeniowe.

settlements_routes.py Rozliczenia, faktury, historia i statusy.

Frontend

Katalog: frontend.

Najważniejsze pliki:

- frontend/src/main.js - start aplikacji,
- frontend/src/router/index.js - routing SPA,
- frontend/src/App.vue - główny kontener,
- frontend/src/composables/useAuth.js - stan zalogowanego użytkownika,
- frontend/src/composables/useToast.js - komunikaty UI.

Główne widoki:

- HomePage.vue - panel użytkownika i dashboard,
- AddedItems.vue - katalog przedmiotów,
- AddedShopPurchaseLists.vue i ShopPurchaseListDetails.vue - listy zakupów,
- PublicPurchasePlans.vue - dofinansowania, budżety i plany,
- PurchaseRequest.vue - wnioski,
- Settlement.vue - rozliczenia i faktury,
- Shops.vue - sklepy.

Przepływ requestu

```
komponent Vue
|
| fetch("http://localhost:8080/api/...")
v
endpoint FastAPI
|
| Pydantic BaseModel / query params
```

```
v
Session(engine)
|
| select / get / add / delete / commit
v
PostgreSQL
|
| response_model albo dict
v
JSON w przeglądarce
```

Konfiguracja środowiska

Backend wymaga DATABASE_URL. W docker-compose.yml baza jest dostępna dla backendu jako:

```
postgresql://budgetflowfusion:budgetflowfusion@db:5432/budgetflowfusion
```

Porty:

```
8080  backend FastAPI
5431  PostgreSQL wystawiony na hosta
```

Inicjalizacja bazy

Przy starcie backend:

1. tworzy tabele przez `SQLModel.metadata.create_all(engine)`,
2. wykonuje `migrate_project_budgets()` dla PostgreSQL,
3. próbuje załadować `backend/scripts/mockup_data.sql`.

Migracja w `src.__init__` dodaje brakujące kolumny i tabele używane przez nowsze funkcje: alokacje finansowania, pozycje planu wniosku, snapshot finalizacji, linie rozliczeniowe i zadania dofinansowań.

Ograniczenia architektury

- Brak oddzielnej warstwy serwisowej: logika domenowa znajduje się w trasach.
- Autoryzacja jest uproszczona: frontend przechowuje użytkownika w `localStorage`, a backend często przyjmuje identyfikatory w payloadzie lub query string.
- Część endpointów zwraca modele `SQLModel` bez dedykowanego DTO.
- Testy używają `SQLite`, a runtime używa `PostgreSQL`.

Model danych

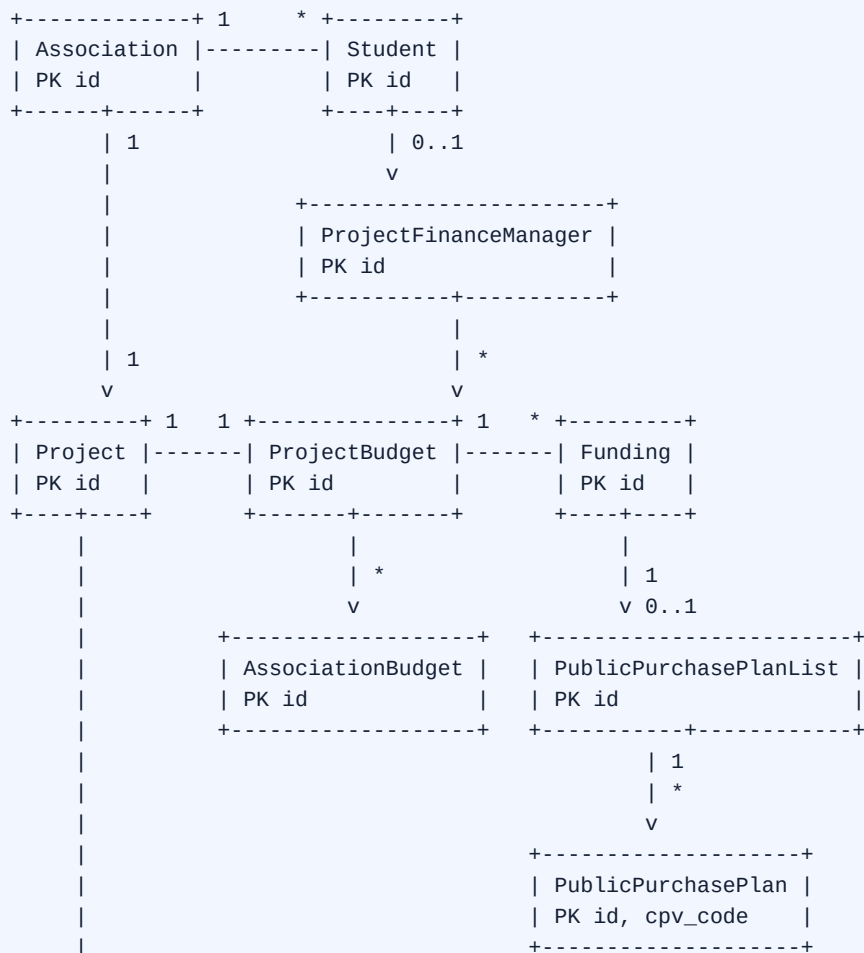
Źródło modelu

Model bazodanowy jest zdefiniowany w backend/src/relations.py jako klasy SQLAlchemy. Tabele są tworzone przez SQLAlchemy.metadata.create_all(engine). Dodatkowe zmiany dla istniejącej bazy PostgreSQL są wykonywane w migrate_project_budgets().

Legenda diagramów

PK klucz główny
FK klucz obcy
1 dokładnie jeden
0..1 zero albo jeden
* wiele

Diagram głównych encji



Opis głównych encji organizacyjno-finansowych

Association Koło naukowe. Grupuje studentów i projekty.

Student Konto użytkownika. Przechowuje dane osobowe, login, hash hasła, przynależność do

koła i opcjonalne powiązanie ze skarbnikiem.

ProjectFinanceManager Konto/rola skarbnika. Powiązanie ze studentem nadaje uprawnienia skarbnika.

Project Sekcja lub projekt koła. Należy do Association.

ProjectBudget Budżet projektu. Ma relację 1:1 z Project i należy do AssociationBudget.

AssociationBudget Budżet koła. Grupuje budżety projektów i dofinansowania.

Funding Dofinansowanie. Należy do budżetu projektu i budżetu koła. Jest głównym źródłem pieniędzy dla list zakupów, planów i wniosków.

FundingTask Zadanie budżetowe w ramach dofinansowania. Suma zadań nie może przekraczać kwoty dofinansowania.

Diagram planu publicznego i wniosku

```
Funding
  | 1
  | 0..1
  v
PublicPurchasePlanList
  | 1
  | *
  v
PublicPurchasePlan
  ^
  | *
  | przez PurchaseRequestPlanPosition
  |
PurchaseRequest
  |
  | * przez PurchaseRequestFundingAllocation
  v
Funding

PurchaseRequest
  |
  +-- PurchaseRequestFinalizationSnapshot
  +-- PurchaseRequestSettlementLine
  +-- Settlement
```

Opis encji planu i wniosku

PublicPurchasePlanList Nagłówek planu publicznego. Ma rok, numer planu, osobę odpowiedzialną, kurs euro i relację 1:1 z Funding.

PublicPurchasePlan Pozycja planu publicznego. Przechowuje nazwę, kod CPV, numer pozycji i planowaną kwotę.

PurchaseRequest Wniosek o zamówienie. Łączy budżet projektu, dofinansowanie, pozycje planu, grupę list zakupów, status finalizacji i dane dokumentowe.

PurchaseRequestFundingAllocation Tabela asocjacyjna PurchaseRequest -> Funding z kwotą allocated_amount. Pozwala podzielić wniosek między wiele źródeł finansowania.

PurchaseRequestPlanPosition Tabela asocjacyjna PurchaseRequest -> ShopPurchaseList -> PublicPurchasePlan z kwotą wykorzystania pozycji planu.

PurchaseRequestFinalizationSnapshot Historyczny zapis danych finalizacji. Przechowuje kopie

nazw, numerów, CPV i kwot użytych w dokumencie.

PurchaseRequestSettlementLine Linia rozliczeniowa wniosku. Zawiera planowaną i rzeczywistą kwotę brutto oraz opcjonalne powiązanie z fakturą.

Diagram list zakupów

```
GroupedShopsListByCpvCategoryAndFunding
| 1
| *
v
ShopPurchaseList
| * |
| przez ShopPurchaseListItem |
v v
Item Student
^ ^
| |
+-- ShopPurchaseListItemContribution
```

Opis encji zakupowych

GroupedShopsListByCpvCategoryAndFunding Grupa koszyków zakupowych. Łączy listy z tym samym kontekstem CPV i finansowania.

ShopPurchaseList Koszyk dla jednego sklepu. Ma koszt, priorytet, datę utworzenia, finansowanie, sklep, właściciela-studenta i opcjonalne rozliczenie.

ShopPurchaseListItem Pozycja koszyka. Kluczem jest para shop_purchase_list_id i item_id. Pole amount zawiera łączną ilość.

ShopPurchaseListItemContribution Wkład konkretnego studenta w pozycję koszyka. Kluczem jest lista, przedmiot i student.

Item Przedmiot w katalogu. Ma cenę, walutę, link, stawkę VAT, status, podkategorię, autora i sklep.

Shop Sklep. Ma nazwę, link, opinię, status, próg darmowej dostawy i listy zakupów.

ProductCategory Kategoria produktu z kodem CPV.

ProductSubcategory Podkategoria produktu. Może być pending, gdy nie ma przypisanej kategorii.

Diagram rozliczeń

```
PurchaseRequest 0..1
|
v
Settlement
| 1
| *
v
Invoice

Settlement
|
+-- ShopPurchaseList
+-- PurchaseRequestSettlementLine
```

Opis encji rozliczeniowych

Settlement Rozliczenie. Może być powiązane z wnioskiem i skarbnikiem, który opłaca rozliczenie.

Invoice Faktura. Zawiera numer, datę wystawienia, sprzedawcę, NIP, kwoty netto i VAT, status oraz powiązanie z rozliczeniem.

Enumy

Currency PLN, EUR, USD.

InvoiceStatus pending, accepted, rejected, paid, returned, arrived.

ItemStatus draft, pending, approved, rejected.

Najważniejsze relacje

```
Association 1..* Student
Association 1..* Project
Project 1..1 ProjectBudget
AssociationBudget 1..* ProjectBudget
ProjectBudget 1..* Funding
Funding 1..0..1 PublicPurchasePlanList
PublicPurchasePlanList 1..* PublicPurchasePlan
Funding 1..* ShopPurchaseList
Shop 1..* ShopPurchaseList
Student 1..* ShopPurchaseList
ProductCategory 1..* ProductSubcategory
ProductSubcategory 1..* Item
ShopPurchaseList *..* Item przez ShopPurchaseListItem
PurchaseRequest *..* Funding przez PurchaseRequestFundingAllocation
PurchaseRequest *..* PublicPurchasePlan przez PurchaseRequestPlanPosition
PurchaseRequest 1..* PurchaseRequestFinalizationSnapshot
PurchaseRequest 1..* PurchaseRequestSettlementLine
Settlement 1..* Invoice
```

Reguły spójności danych

- Jedno dofinansowanie może mieć najwyżej jeden plan publiczny.
- Pozycje planu muszą mieć dodatni koszt.
- Kod CPV powinien być tekstem, nie liczbą.
- Pozycja planu nie powinna być usuwana, jeśli jest użyta przez listę lub wniosek.
- Wniosek musi wskazywać budżet projektu i finansowanie.
- Alokacja planu musi należeć do finansowania użytego we wniosku.
- Finalizacja ustawia can_add = False i zapisuje snapshot.
- Rozliczenie zakończone wymaga faktur.

Procesy biznesowe

Proces bazowy

1. Skarbnik zakłada dofinansowanie.
2. Skarbnik tworzy plan publiczny dla dofinansowania.
3. Użytkownicy dodają przedmioty i pracują na listach zakupów.
4. Skarbnik zamyka listy zakupów.
5. Skarbnik tworzy wniosek.
6. System rezerwuje budżet i pozycje planu.
7. Skarbnik finalizuje wniosek.
8. System zapisuje snapshot i linie rozliczeniowe.
9. Wniosek przechodzi do rozliczeń.
10. Faktury zamykają rozliczenie.

Dodanie przedmiotu

```
Student
-> POST /api/items
-> Item(status = approved)
-> GET /api/items
```

Reguły:

- wymagane są nazwa, cena, waluta, podkategoria i student,
- przedmiot jest przypisany do sklepu,
- katalog pokazuje rekordy zaakceptowane,
- skarbnik może dodatkowo zatwierdzać albo odrzucać pozycje pending.

Zgłoszenie sklepu

```
Student
-> POST /api/shops
-> Shop(status = pending)
-> Skarbnik
-> PATCH approve albo DELETE reject
```

Reguły:

- nazwa sklepu jest wymagana,
- próg darmowej dostawy nie może być ujemny,
- nazwa i link nie mogą duplikować aktywnego sklepu,
- zaakceptowanego sklepu nie można odrzucić endpointem reject.

Dofinansowanie

```
ProjectBudget
-> Funding
-> FundingTask*
```

Reguły:

- kwota dofinansowania musi być dodatnia,
- wymagane są nazwa, organizator i osoba podpisująca,
- suma zadań budżetowych nie może przekroczyć kwoty dofinansowania,
- dofinansowanie dziedziczy project_id i association_budget_id z budżetu projektu.

Plan zamówień publicznych

```
Funding
-> PublicPurchasePlanList
-> PublicPurchasePlan*
```

Reguły:

- jedno dofinansowanie ma najwyżej jeden nagłówek planu,
- pozycja planu ma kod CPV i dodatnią kwotę,
- kod CPV powinien być unikalny w ramach planu,
- usunięcie pozycji jest blokowane, jeżeli pozycja jest użyta przez wniosek lub koszyk.

Lista zakupów

```
ShopPurchaseList
-> ShopPurchaseListItem
-> ShopPurchaseListItemContribution
```

Reguły:

- lista jest przypisana do sklepu, finansowania i studenta,
- pozycja listy przechowuje ilość łączną,
- contribution przechowuje ilość dodaną przez konkretnego studenta,
- zwykły członek może usunąć tylko swój wkład,
- zamknięcie listy tworzy albo wiąże Settlement.

Wniosek

```
POST /api/create_purchase_requests
|
+-- walidacja budżetu projektu
+-- walidacja dofinansowania
+-- walidacja pozycji planu
+-- zapis PurchaseRequest
+-- zapis PurchaseRequestFundingAllocation
+-- zapis PurchaseRequestPlanPosition
```

Reguły:

- funding_id musi należeć do wskazanego project_budget_id,
- kwota wniosku nie może przekroczyć dostępnych środków projektu,
- kwota wniosku nie może przekroczyć dostępnych środków finansowania,
- pozycja planu musi należeć do finansowania użytego we wniosku,
- przekroczenie pozostałej kwoty planu ustawia plan_compliance_status = requires_approval,

- status requires_approval wymaga uzasadnienia.

Wniosek z zamkniętej listy

```
PATCH /api/lists/{id}/close
-> Settlement bez purchase_request_id
-> ShopPurchaseList.settlement_id

POST /api/create_purchase_requests
shop_purchase_list_id = id
-> PurchaseRequest
-> Settlement.purchase_request_id = request.id
```

Reguły:

- lista musi być zamknięta,
- lista musi należeć do tego samego skarbnika,
- settlement listy nie może być już powiązany z innym wnioskiem,
- koszt wniosku jest liczony z pozycji listy.

Finalizacja

```
prepare_finalization
-> can_add = False
-> finalization_status = prepared

finalize
-> PurchaseRequestFinalizationSnapshot*
-> PurchaseRequestFundingAllocation*
-> PurchaseRequestSettlementLine*
-> finalization_status = accounting_pending
```

Reguły:

- wniosek musi mieć koszyki,
- wniosek musi mieć pozycje CPV,
- kurs euro musi być dodatni,
- nazwa dokumentu jest wymagana,
- finalizacja zapisuje snapshot i wartości końcowe netto/brutto,
- po finalizacji nie można dalej dodawać pozycji do wniosku.

Rozliczenie

```
send_to_settlement
-> finalization_status = settlement
-> Settlement
-> Invoice*
-> complete
-> finalization_status = settled
```

Reguły:

- do rozliczeń można przekazać tylko zatwierdzony wniosek,
- dodatkowe linie rozliczeniowe można dodać dopiero po przekazaniu do rozliczeń,

- zakończenie rozliczenia wymaga faktur,
- linie rozliczeniowe pokazują różnicę między kwotą planowaną i rzeczywistą.

API backendu

Informacje ogólne

Base URL:

```
http://localhost:8080/api
```

Format danych: JSON.

Backend używa FastAPI, Pydantic i SQLAlchemy. Sesja bazy jest przekazywana do endpointów przez Depends(get_session).

Autoryzacja

Autoryzacja jest uproszczona. Frontend przechowuje użytkownika w localStorage. Backend w wielu miejscach przyjmuje student_id, association_id albo project_finance_manager_id w query string lub payloadzie. Wybrane endpointy sprawdzają, czy student ma powiązanie z ProjectFinanceManager.

Logowanie

POST /api/login Loguje użytkownika.

POST /api/register Rejestruje użytkownika i opcjonalnie tworzy rolę skarbnika.

Użytkownicy

GET /api/students Lista studentów.

Kategorie

GET /api/categories Lista kategorii.

POST /api/categories Tworzy kategorię. Wymaga skarbnika.

GET /api/subcategories Lista podkategorii przypisanych do kategorii.

GET /api/subcategories/pending Lista podkategorii bez przypisanej kategorii.

POST /api/subcategories Tworzy podkategorię. Przypisanie do kategorii wymaga skarbnika.

PATCH /api/subcategories/{subcategory_id}/assign-category Przypisuje podkategorię do kategorii. Wymaga skarbnika.

Przedmioty

GET /api/items Lista zaakceptowanych przedmiotów.

GET /api/items/grouped?group_by=shop|cpv|category|status Lista przedmiotów pogrupowanych.

GET /api/items/pending Przedmioty oczekujące. Może filtrować po association_id.

POST /api/items Tworzy przedmiot.

PATCH /api/items/{item_id}/approve Zatwierdza przedmiot.

DELETE /api/items/{item_id}/reject Odrzuca przedmiot.

Sklepy

GET /api/shops?include_pending=false Lista sklepów. Domyślnie tylko zaakceptowane.

POST /api/shops Zgłasza sklep.

PATCH /api/shops/{shop_id} Edytuje sklep. Wymaga skarbnika.

PATCH /api/shops/{shop_id}/approve Akceptuje sklep. Wymaga skarbnika.

DELETE /api/shops/{shop_id}/reject?student_id=... Odrzuca sklep oczekujący. Wymaga skarbnika.

Dofinansowania i budżety

GET /api/fundings?association_id=... Lista dofinansowań z wyliczonymi kwotami dostępnymi.

POST /api/fundings Tworzy dofinansowanie i zadania budżetowe.

PATCH /api/fundings/{funding_id} Aktualizuje dofinansowanie i zastępuje jego zadania.

GET /api/association_budgets?association_id=... Budżety kół.

GET /api/project_budgets?association_id=... Budżety projektów/sekcji.

GET /api/dashboard/budget_summary?association_id=... Agregat budżetowy dla pulpitu.

Plany publiczne

POST /api/public_purchase_plan_lists Tworzy albo aktualizuje plan publiczny dla dofinansowania.

GET /api/public_purchase_plan_lists Lista planów. Obsługuje filtry association_id i funding_id.

GET /api/public_purchase_plan_lists/{public_purchase_plan_list_id} Szczegóły planu.

POST /api/public_purchase_plans Dodaje pozycję planu.

PATCH /api/public_purchase_plans/{public_purchase_plan_id} Aktualizuje pozycję planu.

DELETE /api/public_purchase_plans/{public_purchase_plan_id} Usuwa pozycję planu, jeśli nie jest użyta.

Listy zakupów

GET /api/lists Lista koszyków. Endpoint obsługuje filtry użytkownika, koła, sklepu, planu, wniosku, grupy i statusu otwarcia.

POST /api/lists Tworzy listę zakupów.

PATCH /api/lists/{list_id}/close Zamyka listę i tworzy albo wiąże rozliczenie.

PATCH /api/lists/{list_id}/reopen Otwiera listę ponownie.

GET /api/lists/closed_for_purchase_requests Zamknięte listy dostępne do utworzenia wniosku.

GET /api/lists/{list_id} Szczegóły listy.

PATCH /api/lists/{list_id}/market_research Zapisuje komentarz i nazwę pliku rozeznania rynku.

DELETE /api/lists/{list_id} Usuwa listę.

GET /api/lists/{list_id}/items Pozycje listy.

POST /api/lists/{list_id}/items Dodaje przedmiot do listy.

PUT /api/lists/{list_id}/items/{item_id} Aktualizuje ilość pozycji.

DELETE /api/lists/{list_id}/items/{item_id} Usuwa pozycję albo wkład użytkownika.

Wnioski

GET /api/purchase_requests?association_id=... Lista wniosków.

GET /api/purchase_requests/detail/{purchase_request_id} Szczegóły jednego wniosku.

GET /api/purchase_requests/{project_finance_manager_id} Wnioski skarbnika.

POST /api/create_purchase_requests Tworzy wniosek.

PATCH /api/purchase_requests/{purchase_request_id} Aktualizuje wniosek.

DELETE /api/purchase_requests/{purchase_request_id} Usuwa wniosek i odpina powiązane rozliczenia.

Finalizacja wniosku

POST /api/purchase_requests/{purchase_request_id}/prepare_finalization Przygotowuje finalizację, blokuje dodawanie i wiąże rozliczenia z listami.

PATCH /api/purchase_requests/{purchase_request_id}/finalization_draft Zapisuje roboczą nazwę dokumentu, kurs euro i datę wartości umowy.

POST /api/purchase_requests/{purchase_request_id}/finalize Finalizuje wniosek, zapisuje snapshoty i ustawia accounting_pending.

POST /api/purchase_requests/{purchase_request_id}/send_to_settlement Przekazuje wniosek do rozliczeń.

POST /api/purchase_requests/{purchase_request_id}/return_to_open Cofa wniosek ze statusu prepared do draft.

Linie rozliczeniowe wniosku

GET /api/purchase_requests/{purchase_request_id}/settlement_lines Lista linii rozliczeniowych.

PUT /api/purchase_requests/{purchase_request_id}/settlement_lines Zapisuje zestaw linii przed przekazaniem do rozliczeń.

PATCH /api/purchase_request_settlement_lines/{settlement_line_id} Aktualizuje pojedynczą linię.

POST /api/purchase_requests/{purchase_request_id}/settlement_lines/extra Dodaje dodatkową linię po przekazaniu do rozliczeń.

Rozliczenia i faktury

GET /api/settlements Lista rozliczeń.

GET /api/settlements/history Rozliczenia zakończone.

GET /api/settlements/manager/{project_finance_manager_id} Rozliczenia skarbnika.

POST /api/settlements Tworzy rozliczenie.

GET /api/invoices Lista faktur. Obsługuje filtry statusu, skarbnika, wniosku i koła.

POST /api/settlements/{settlement_id}/invoices Dodaje fakturę do rozliczenia.

PATCH /api/invoices/{invoice_id} Aktualizuje fakturę.

POST /api/settlements/{settlement_id}/complete Kończy rozliczenie.

PATCH /api/settlements/{settlement_id}/status Zmienia status rozliczenia/faktur według

logiki endpointu.

Statusy

Statusy wniosku:

```
draft
prepared
accounting_pending
settlement
settled
```

Statusy zgodności planu:

```
draft
compliant
requires_approval
```

Statusy faktury:

```
pending
accepted
rejected
paid
returned
arrived
```

Dokumentacja użytkownika

Przeznaczenie

Aplikacja służy do prowadzenia zakupów koła naukowego od zgłoszenia przedmiotu do rozliczenia faktury. Użytkownik pracuje w przeglądarce. Dane są zapisywane na backendzie.

Role użytkowników

Zwykły członek może:

- logować się i przeglądać swój profil,
- dodawać przedmioty,
- zgłaszać sklepy,
- przeglądać dostępne listy zakupów,
- dodawać przedmioty do otwartych list,
- usuwać własny wkład z listy.

Skarbnik może dodatkowo:

- zarządzać sklepami i kategoriami,
- tworzyć dofinansowania,
- tworzyć plany zamówień publicznych,
- tworzyć i zamykać listy zakupów,
- tworzyć wnioski,
- finalizować wnioski,
- przekazywać wnioski do rozliczeń,
- dodawać faktury,
- kończyć rozliczenia,
- kontrolować budżet koła i sekcji.

Logowanie

1. Otwórz aplikację.
2. Wpisz login/e-mail i hasło.
3. Zatwierdź formularz.
4. Po poprawnym logowaniu aplikacja pokazuje panel główny.

Rejestracja

1. Otwórz formularz rejestracji.
2. Podaj imię, nazwisko, login/e-mail i hasło.
3. Wybierz koło i sekcję/projekt.
4. Ustaw informację SAP.
5. Wybierz rolę.
6. Zapisz konto.

Status SAP jest informacyjny. Aplikacja nie łączy się z SAP.

Panel główny

Panel pokazuje dane użytkownika:

- imię i nazwisko,
- login/e-mail,
- koło,
- sekcję,
- rolę,
- status SAP.

Skarbnik widzi dodatkowo podsumowanie budżetu:

- budżet całkowity,
- wydane środki,
- środki zarezerwowane we wnioskach,
- środki dostępne po rezerwacjach,
- procent wykorzystania.

Katalog przedmiotów

Dodanie przedmiotu:

1. Przejdź do widoku dodanych przedmiotów.
2. Wybierz dodanie nowego przedmiotu.
3. Podaj nazwę, link, cenę, walutę, sklep i podkategorię.
4. Zapisz formularz.

Przedmiot trafia do katalogu i może zostać użyty na liście zakupów.

Kategorie i podkategorie

Skarbnik może tworzyć kategorie z kodem CPV. Użytkownik może zgłosić podkategorię.

Podkategoria bez kategorii trafia do oczekujących i może zostać przypisana przez skarbnika.

Sklepy

Zgłoszenie sklepu:

1. Przejdź do widoku sklepów.
2. Dodaj sklep.
3. Podaj nazwę, link, opinię i próg darmowej dostawy.
4. Zapisz.

Skarbnik może:

- zaakceptować sklep,
- edytować sklep,
- odrzucić sklep oczekujący.

Listy zakupów

Lista zakupów jest koszykiem dla jednego sklepu.

Stany listy:

otwarta - można dodawać i usuwać pozycje
zamknięta - nie można zmieniać pozycji, można użyć we wniosku

Dodanie pozycji do listy:

1. Otwórz listę.
2. Wybierz przedmiot z katalogu.
3. Podaj ilość.
4. Zapisz.

Jeżeli kilku użytkowników dodaje ten sam przedmiot, aplikacja pokazuje łączną ilość, ale pamięta wkład każdego użytkownika oddzielnie.

Zamknięcie listy:

1. Otwórz listę jako skarbnik.
2. Sprawdź pozycje i koszt.
3. Zamknij listę.

Zamknięta lista może zostać użyta do utworzenia wniosku.

Dofinansowania

Tworzenie dofinansowania:

1. Przejdź do widoku planów publicznych/budżetów.
2. Wybierz sekcję/projekt.
3. Podaj nazwę dofinansowania.
4. Podaj organizatora.
5. Podaj osobę podpisującą.
6. Podaj termin wydatkowania.
7. Podaj kwotę.
8. Opcjonalnie dodaj zadania budżetowe.
9. Zapisz.

Suma zadań budżetowych nie może przekroczyć kwoty dofinansowania.

Plany publiczne

Utworzenie planu:

1. Wybierz dofinansowanie.
2. Utwórz plan.
3. Podaj rok, numer planu, osobę odpowiedzialną i kurs euro.
4. Zapisz.

Dodanie pozycji planu:

1. Otwórz plan.
2. Dodaj pozycję.
3. Podaj nazwę, CPV, numer pozycji i kwotę.
4. Zapisz.

Znaczenie kwot:

kwota planowana	- limit pozycji planu
wykorzystano	- suma alokacji z wniosków
pozostało	- kwota planowana minus wykorzystanie

Wnioski

Utworzenie wniosku:

1. Przejdź do widoku wniosków.
2. Utwórz wniosek albo wybierz zamkniętą listę.
3. Podaj nazwę i typ zakupu.
4. Wybierz finansowanie.
5. Wskaż pozycje planu CPV i kwoty.
6. Jeżeli system wymaga uzasadnienia, uzupełnij powód odstąpienia.
7. Zapisz.

Po zapisaniu wniosków rezerwuje środki. Budżet dostępny zmniejsza się o kwotę alokacji.

Finalizacja wniosku

1. Otwórz wniosek.
2. Przygotuj finalizację.
3. Sprawdź koszyki, kwoty i CPV.
4. Podaj nazwę dokumentu.
5. Podaj datę wartości umowy.
6. Podaj kurs euro.
7. Finalizuj.

Po finalizacji:

- dodawanie pozycji jest zablokowane,
- zapisywany jest snapshot danych,
- tworzone są linie rozliczeniowe,
- wniosek przechodzi do oczekiwania na księgowość.

Rozliczenia i faktury

Przekazanie do rozliczeń:

1. Otwórz sfinalizowany wniosek.
2. Wybierz przekazanie do rozliczeń.
3. Sprawdź linie rozliczeniowe.

Dodanie faktury:

1. Otwórz rozliczenie.
2. Dodaj fakturę.
3. Podaj numer, datę, sprzedawcę, NIP, kwotę netto i VAT.
4. Zapisz.

Zakończenie rozliczenia:

1. Uzupełnij faktury.
2. Przypisz kwoty rzeczywiste do linii.

3. Zakończ rozliczenie.

Historia

Historia rozliczeń pokazuje wnioski zakończone. Widok służy do kontroli faktur, kwot rzeczywistych i różnic względem planu.

Uruchomienie i testy

Wymagania

Do uruchomienia projektu potrzebne są:

- Docker,
- Docker Compose,
- Node.js i npm,
- przeglądarka internetowa.

Backend i baza

Uruchomienie kontenerów:

```
docker compose up --build
```

Adres backendu:

```
http://localhost:8080
```

PostgreSQL jest wystawiony lokalnie na porcie:

```
5431
```

Reset bazy:

```
docker compose down -v  
docker compose up --build
```

Po starcie backend tworzy tabele, wykonuje migrację i próbuje załadować:

```
backend/scripts/mockup_data.sql
```

Frontend

Instalacja zależności:

```
cd frontend  
npm install
```

Tryb developerski:

```
npm run dev
```

Build:

```
npm run build
```

Testy backendu

Katalog testów:

```
backend/tests
```

Uruchomienie całego pakietu w kontenerze backendu:

```
docker exec -it backend python -m pytest tests/ -v
```

Uruchomienie pojedynczego pliku:

```
docker exec -it backend python -m pytest tests/test_purchase_request.py -v
```

Uruchomienie jednego testu:

```
docker exec -it backend python -m pytest  
tests/test_purchase_request.py::test_create_purchase_request -v
```

Tryb po ostatniej awarii:

```
docker exec -it backend python -m pytest tests/ --lf -v
```

Testowa baza danych

Testy backendu używają SQLite in-memory. Fixture z `conftest.py` podmienia zależność `get_session` i tworzy tabele z modeli `SQLModel`.

Konsekwencje:

- testy są szybkie,
- testy nie wymagają PostgreSQL,
- testy nie pokrywają w pełni różnic dialektu PostgreSQL,
- po zmianach migracji trzeba sprawdzić również start kontenerów.

Zakres testów

`test_business_logic.py` Kategorie, podkategorie, katalog przedmiotów, akceptacja, odrzucanie i izolacja danych między kołami.

`test_public_purchase_plans.py` Budżety, dofinansowania, dashboard, plany publiczne, CPV i walidacja kwot.

`test_purchase_request.py` Tworzenie wniosków, alokacje, plan publiczny, przekroczenia planu, usuwanie i aktualizacje.

`test_settlements.py` Rozliczenia, faktury, filtrowanie, blokady i statusy.

Generowanie dokumentacji

Dokumentacja jest generowana skryptem:

```
python3 doc_gen.py
```

Wyniki:

```
doc/build/html/index.html  
doc/build/pdf/BudgetFlowFusion_documentation.pdf
```

Skrypt używa tylko standardowej biblioteki Pythona.

Utrzymanie i słownik pojęć

Zasady utrzymania

1. Zmiana modelu w relations.py wymaga sprawdzenia migracji w src/__init__.py.
2. Zmiana endpointu wymaga aktualizacji dokumentacji API.
3. Zmiana procesu użytkownika wymaga aktualizacji dokumentacji użytkowej.
4. Zmiana budżetów wymaga testów dofinansowań, dashboardu, wniosków i rozliczeń.
5. Zmiana planów publicznych wymaga testów wykorzystania CPV i finalizacji.
6. Zmiana finalizacji wymaga sprawdzenia snapshotów i linii rozliczeniowych.

Miejsca wysokiego ryzyka

Budżety Kwoty są liczone z Funding, PurchaseRequestFundingAllocation i Funding.spent_money. Należy unikać podwójnego liczenia.

Plany publiczne Pozycje planu są używane przez wnioski i listy. Usuwanie lub edycja pozycji wpływa na raportowanie wykorzystania.

Finalizacja Snapshot jest historią dokumentu. Nie powinien być kasowany poza świadomą operacją ponownej finalizacji.

Rozliczenia Faktury, settlementy i linie rozliczeniowe są powiązane z wnioskiem. Zmiana statusu powinna zachować historię kwot.

Autoryzacja Frontend ukrywa część funkcji, ale backend powinien walidować operacje finansowe niezależnie od UI.

Checklist: zmiana modelu danych

1. Zmień klasę SQLModel w backend/src/relations.py.
2. Dodaj migrację PostgreSQL w migrate_project_budgets(), jeżeli baza może już istnieć.
3. Zaktualizuj modele Pydantic w trasach.
4. Zaktualizuj frontendowe payloady i odczyty odpowiedzi.
5. Zaktualizuj testy fixture.
6. Uruchom testy backendu.
7. Uruchom backend z PostgreSQL.
8. Zaktualizuj dokumentację.

Checklist: zmiana budżetów

Sprawdź:

- public_purchase_plans_routes.py - agregaty budżetowe,
- fundings_routes.py - dostępne środki dofinansowania,
- purchase_request_routes.py - rezerwacje i finalizacja,
- dashboard frontendu,
- testy public purchase plans i purchase request.

Minimalny scenariusz ręczny:

1. Utwórz dofinansowanie.

2. Sprawdź budżet sekcji.
3. Sprawdź budżet koła.
4. Utwórz wniosek.
5. Sprawdź spadek dostępnych środków.
6. Sfinalizuj wniosek.
7. Sprawdź rozliczenie.

Checklist: zmiana planów publicznych

Sprawdź:

- jeden plan dla jednego dofinansowania,
- dodatnie kwoty pozycji,
- unikalność CPV w planie,
- remaining_amount,
- blokadę usunięcia użytej pozycji,
- walidację pozycji planu przy wniosku,
- snapshot finalizacji.

Checklist: zmiana wniosków

Sprawdź:

- zgodność funding_id z project_budget_id,
- alokacje finansowania,
- alokacje pozycji planu,
- uzasadnienie odstępstwa,
- przejścia statusów,
- finalizację,
- przekazanie do rozliczeń,
- usunięcie wniosku z odpięciem settlementów.

Słownik

Koło naukowe Association.

Członek koła Student bez powiązania z ProjectFinanceManager.

Skarbnik Student z ustawionym project_finance_manager_id.

Sekcja/projekt Project.

Budżet sekcji ProjectBudget.

Budżet koła AssociationBudget.

Dofinansowanie Funding.

Zadanie budżetowe FundingTask.

Plan publiczny PublicPurchasePlanList.

Pozycja planu PublicPurchasePlan.

Koszyk/lista zakupów ShopPurchaseList.

Pozycja koszyka ShopPurchaseListItem.

Wkład użytkownika ShopPurchaseListItemContribution.

Wniosek PurchaseRequest.

Alokacja finansowania PurchaseRequestFundingAllocation.

Alokacja planu PurchaseRequestPlanPosition.

Snapshot finalizacji PurchaseRequestFinalizationSnapshot.

Rozliczenie Settlement.

Faktura Invoice.

Linia rozliczeniowa PurchaseRequestSettlementLine.